COMP438/538 Reinforcement Learning

Model-Free Control

Barış Akgün

Fall 2026

What changes when we want better behaviour?

- With a model: evaluate a policy, then improve it.
- Last week: evaluate a **fixed** policy from experience.
- Today: use experience to **improve the policy too**.

Question: Which half of generalized policy iteration did we hold fixed last week?

Why Model-Free Control?

For a lot of real-world problems that can be modelled by MDPs:

- Samples can be obtained (even if costly)
- Models are unknown

OR

- Models are known but they are too computationally expensive to be used (for RL e.g. due to high dimensionality, not for sampling)
- Learning models are not realistic (e.g. too complex, high dimensional) and/or sampling is not costly (models not really needed; we have the real world ☺)

How could we learn to act without the model?

1. What should the agent learn so it can choose better actions?

2. How do its choices affect the data it will see?

Today's idea:

- Estimate Q-Values instead of values
- Try different actions



No model. The agent now chooses its actions.

Why learn action values?

With state values, greedy improvement needs the model:

$$\pi'(s) = \arg\max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') v^{\pi}(s')]$$

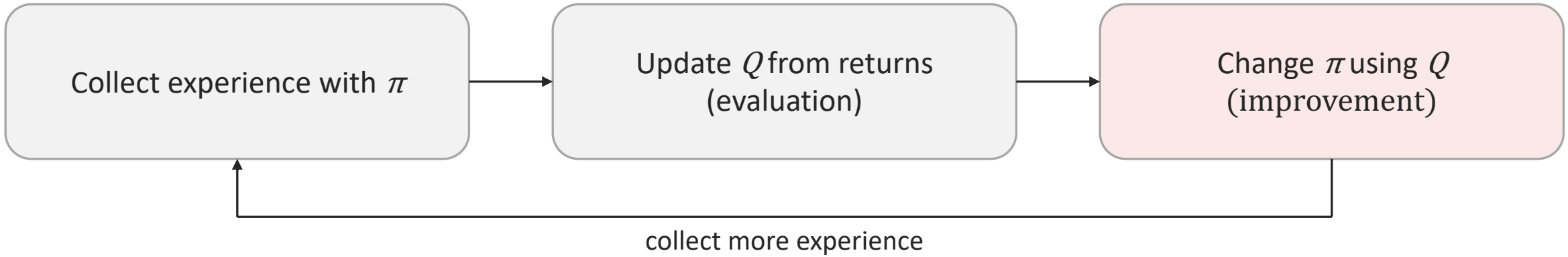
With learned action values, choose directly from the table:

$$\pi'(s) = \arg\max_{a \in \mathcal{A}(s)} Q(s,a)$$

$q^{\pi}(s,a)$: **take action a now, then follow π.**

$Q(s,a)$: **our estimate. We need evidence about actions, not just states.**

Evaluation and improvement interleave



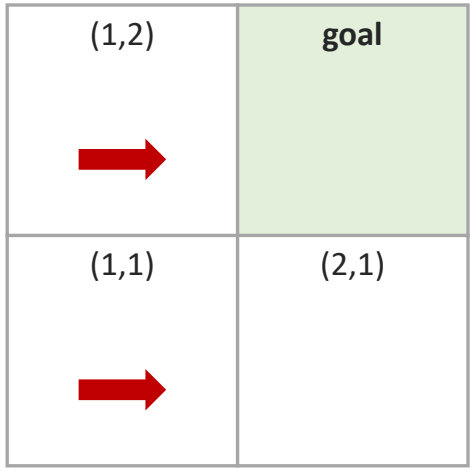
- Evaluation changes the estimates.
- Improvement changes the policy being evaluated.
- We need not finish evaluation before changing the policy.

One noisy update is not a guarantee of improvement.

Trace one Monte Carlo control cycle

New Q table, initially zero. Hold the collecting policy fixed during this episode:

(1,1) → (1,1) → (1,2) → goal, taking **right** each time; $\gamma = 0.5$.



Time	State-action pair	Reward	Supplied return
0	((1,1), right)	−0.04	0.19
1	((1,1), right)	−0.04	0.46
2	((1,2), right)	+1	1

First-visit averaging: which returns enter the table?

$Q((1,1), \text{right}) = 0.19$; $Q((1,2), \text{right}) = 1$.

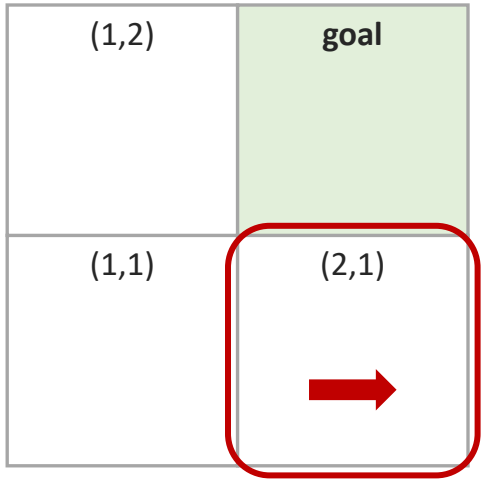
The repeated first pair is skipped and unvisited entries remain zero.

Side Question: What does the table look-like?

Would pure greediness find the better action?

Separate hypothetical snapshot at (2,1)

Action	up	right	down	left
Current Q	0	0.12	0	0
Tried here?	no	yes	no	no



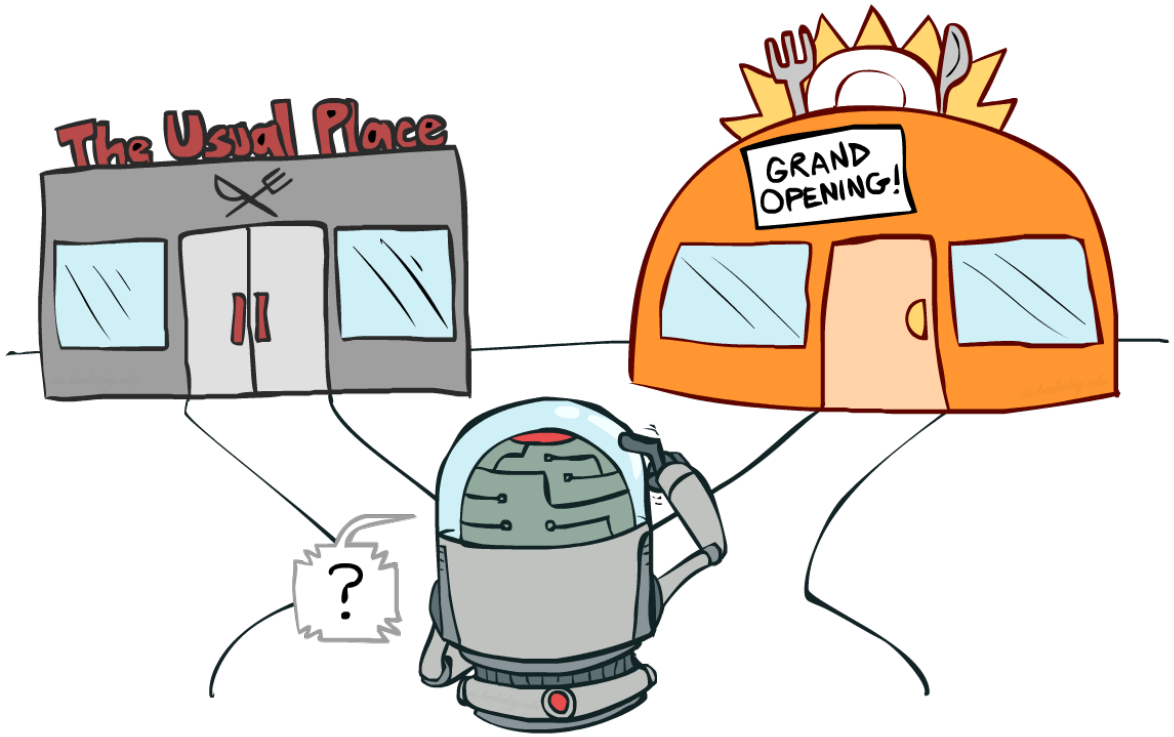
The greedy choice is right. What would make the learner try up?

Known-model reference from Slide Deck 2, not supplied to the learner (π = always-right thereafter):

$q^{\pi}((2,1), \text{right}) \approx 0.116$, $q^{\pi}((2,1), \text{up}) \approx 0.800$

An untried action's initial estimate is not evidence that it is poor.

Exploration and exploitation



Exploit: choose what current estimates say is good.

Explore: obtain evidence that could change those estimates.

Ways to obtain action coverage:

- Start episodes at different state-action pairs, when the system allows it.
- Keep a chance of trying other actions while acting.

Epsilon-greedy action selection

- With probability **1 – ε**, choose a maximizing action; break ties uniformly at random.
- With probability **ε**, choose uniformly from **all** available actions.

For a unique maximizing action:

The greedy action: $\pi(a \mid s) = 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)|$

Every other action: $\pi(a \mid s) = \varepsilon/|\mathcal{A}(s)|$

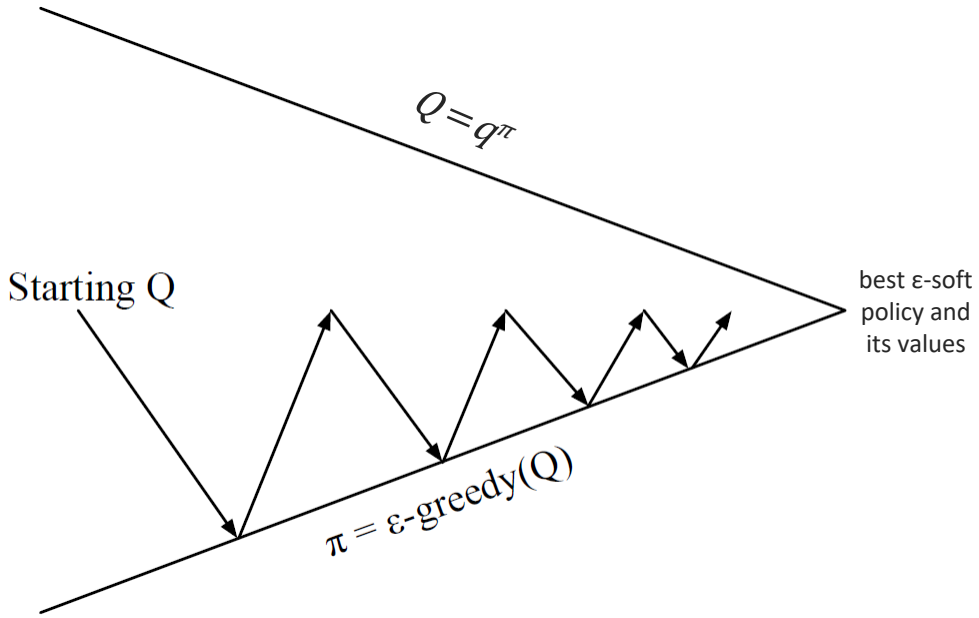
- **ε = 1:** uniform random actions: maximal exploration.
- **ε = 0:** pure greedy: no exploration (the lock-in from slide 9).
- **In practice:** start exploratory and decay ε slowly, so every state–action pair keeps receiving data.

Question: Four actions, ε = 0.1, one maximum: what are the probabilities?

0.925 for the maximizing action; 0.025 for each other action.

On-policy First-Visit Monte Carlo control

1. Initialize Q and an epsilon-greedy policy, $\varepsilon = 0.1$.
2. Generate a complete episode with this policy held fixed.
3. For each pair's first visit, add its return to that pair's samples and set Q to their average.
4. Make the next episode's policy epsilon-greedy with respect to the updated Q.
5. Repeat.



Exact-evaluation illustration with ε fixed, not a trace of the sampled loop.

Exact-value guide: epsilon-greedy improvement using true q^{π} is no worse than the epsilon-soft policy π being evaluated.

Sampled updates can be noisy. They do not inherit a per-update guarantee.

Best ε-soft policy: the best among policies that keep exploring at rate ε, not π^*.

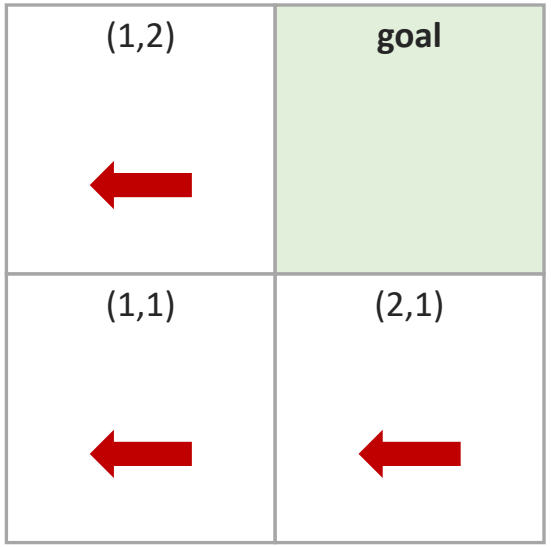
Question: what if we decayed ε?

Decay slowly, with every state-action pair still tried infinitely often: it converges to q^* and π^*.

Why wait for the episode to end?

Counterexample: deterministic always-left, ε = 0.

Starting at (1,1), the agent stays in the left column and never reaches the goal.



MC needs a completed return. TD can update during the episode.

With fixed $\varepsilon > 0$, this grid terminates with probability 1 but MC still waits for completion.

SARSA: a sampled action-value target

For a fixed π, recall the action-value Bellman expectation equation:

$$q^{\pi}(s,a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma q^{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Observe the next transition, choose the next action using π, and use current estimates:

$$U_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}), \qquad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[U_t - Q(S_t, A_t)]$$

S–A–R–S–A: $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

Terminal next state: $U_t = R_{t+1}$, no next action is needed.

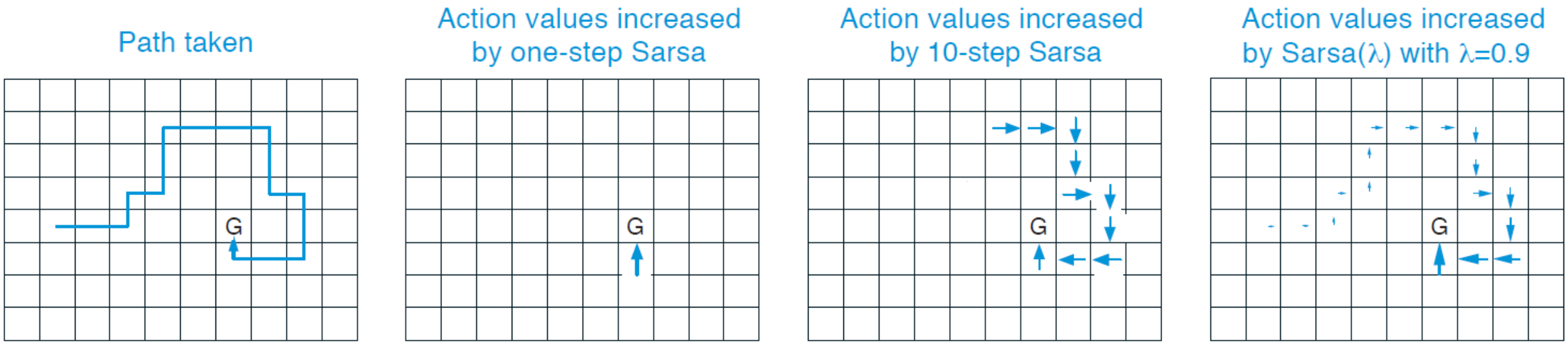
Trace the SARSA loop

Initialize Q for nonterminal pairs. For each episode:

1. Observe S_0; choose A_0 epsilon-greedily from Q.
2. Execute A_t; observe R_{t+1} and S_{t+1}.
3. If terminal: use $U_t = R_{t+1}$. Otherwise **choose** A_{t+1} epsilon-greedily and use $U_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$.
4. Update $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[U_t - Q(S_t, A_t)]$.
5. If terminal, end the episode. Otherwise advance one step and **execute the already-selected** A_{t+1}.

Question: Where are exploration and improvement happening?

Longer SARSA targets: credit further back



Sutton & Barto, Figure 7.4 and Example 12.1 (separate illustration, γ = 1): one episode, all rewards 0 except at G, Q initially 0.

- One-step SARSA increases only the last action's value; a 10-step target, or a λ-mixture of targets, also credits earlier actions.
- Same idea as Week 3's n-step and λ-return prediction targets, with Q in place of V.

This is to make a connection. We are skipping the algorithms, but the book has them.

Q-learning: sampled Q-value iteration

With a model:

$$Q_{k+1}(s,a) \leftarrow \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') \max_{a' \in \mathcal{A}(s')} Q_k(s',a')$$

With one sampled transition:

$$U_t = R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a), \qquad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[U_t - Q(S_t, A_t)]$$

Act exploratorily; compute a greedy continuation in the target.

Q-learning still needs an acting policy

Initialize Q. For each episode:

1. Observe the current nonterminal state S_t.
2. Choose and execute A_t **epsilon-greedily** from Q.
3. Observe R_{t+1}, S_{t+1}.
4. If terminal, $U_t = R_{t+1}$. Otherwise, $U_t = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$.
5. Update $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[U_t - Q(S_t, A_t)]$.
6. If terminal, end the episode; otherwise advance and choose the next **behaviour** action.

The action used in the maximum need not be the action executed next.

Behaviour and target policies

Behaviour b generates actions in the data. Target π describes the continuation being learned about.

Method	Behaviour	Continuation in the target
SARSA	epsilon-greedy	the same acting policy: sampled next action
Q-learning	epsilon-greedy	greedy continuation: maximum next-action value

On-policy (SARSA): learns the values of the policy it is following; the target uses the next action that policy actually chose.

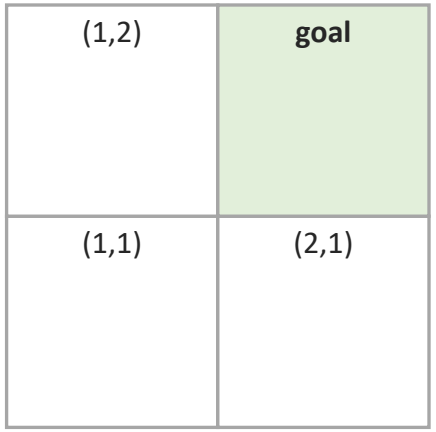
Off-policy (Q-learning): learns about the greedy policy while the data come from an epsilon-greedy policy.

More general off-policy learning needs corrections; we will learn about them later in the course.

Your turn: one transition, two targets

$\gamma = 0.5,\ \alpha = 0.5,\ Q((1,1), \text{right}) = 0.1.$

Action at (1,2)	right	up	down	left
Q	0.8	0.2	0.1	0



Observed transition: **(1,1), right, –0.04, (1,2)**. The next epsilon-greedy action is **up**.

1. Calculate each target and the updated entry for SARSA and Q-learning.
2. Which next action does each target use?
3. When can the two targets agree?

Separate terminal case - reset the exercise: (1,2), right, +1, goal. What is each target?

Compare the updates

	SARSA	Q-learning
Continuation	$Q((1,2),\text{up})=0.2$	$\max Q((1,2),\cdot)=0.8$
U_t	−0.04 + 0.5 × 0.2 = **0.06**	−0.04 + 0.5 × 0.8 = **0.36**
Updated $Q((1,1),\text{right})$	0.1 + 0.5(0.06 − 0.1) = **0.08**	0.1 + 0.5(0.36 − 0.1) = **0.23**

Same snapshot, same observed reward and next state but different continuation.

Separate goal transition: **both targets are 1**, with zero continuation.

What is each update trying to learn?

Method	Update target	Continuation represented	DP connection
MC control	G_t	episode's collecting policy	evaluation/improvement idea; complete return
SARSA	$R_{t+1}+\gamma Q(S_{t+1},A_{t+1})$	behaviour policy	sampled action-value evaluation plus improvement
Q-learning	$R_{t+1}+\gamma \max_a Q(S_{t+1},a)$	greedy continuation	sampled optimality backup

SARSA estimates action values for the policy it follows. Q-learning aims at optimal action values while exploring.

From target differences to behaviour

- Q lets us compare actions without a transition model.
- Exploration determines which estimates receive evidence.
- SARSA and Q-learning use different future-action assumptions.

Can we think of something between SARSA and Q-Learning?

Expected SARSA

A nonterminal transition gives reward 0; $\gamma = 0.5$. At the next state the action values are 5 (greedy), 2, 1 and 0, and the ε-greedy policy (ε = 0.1, four actions) chose the action with value 2.

Compute SARSA's, Q-learning's and an averaged target.

Averaged target (Expected SARSA):

$$U_t = R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a)$$

Method	Next-action value used	Target
SARSA	the sampled next action: 2	0 + 0.5 × 2 = **1**
Expected SARSA	the policy's average: 0.925 × 5 + 0.025 × (2 + 1 + 0) = 4.7	0 + 0.5 × 4.7 = **2.35**
Q-learning	the maximum: 5	0 + 0.5 × 5 = **2.5**

Expected SARSA averages over the next action instead of sampling it.

We can use different target policies in Expected SARSA.

When do these methods converge? (tabular)

Standard sufficient conditions:

- **every state–action pair keeps being updated**
- the step sizes used for each pair's own updates shrink, but not too fast: $\sum \alpha = \infty, \sum \alpha^2 < \infty$ (previous slide deck)

With noisy targets, constant α can leave persistent estimate fluctuations; exact convergence is not generally guaranteed.

- **MC control and SARSA:** converge to q^* and π^* if exploration also fades slowly: greedy in the limit while every pair keeps receiving data (GLIE: Greedy in the Limit with Infinite Exploration). With fixed ε, their on-policy targets include continuing exploration costs; q^* is not generally the target.
- **Q-learning:** converges to q^* without the behaviour becoming greedy (off-policy).

These are tabular guarantees. Q-learning already bootstraps and learns off-policy; add function approximation and convergence can fail: the deadly triad (Next topic).

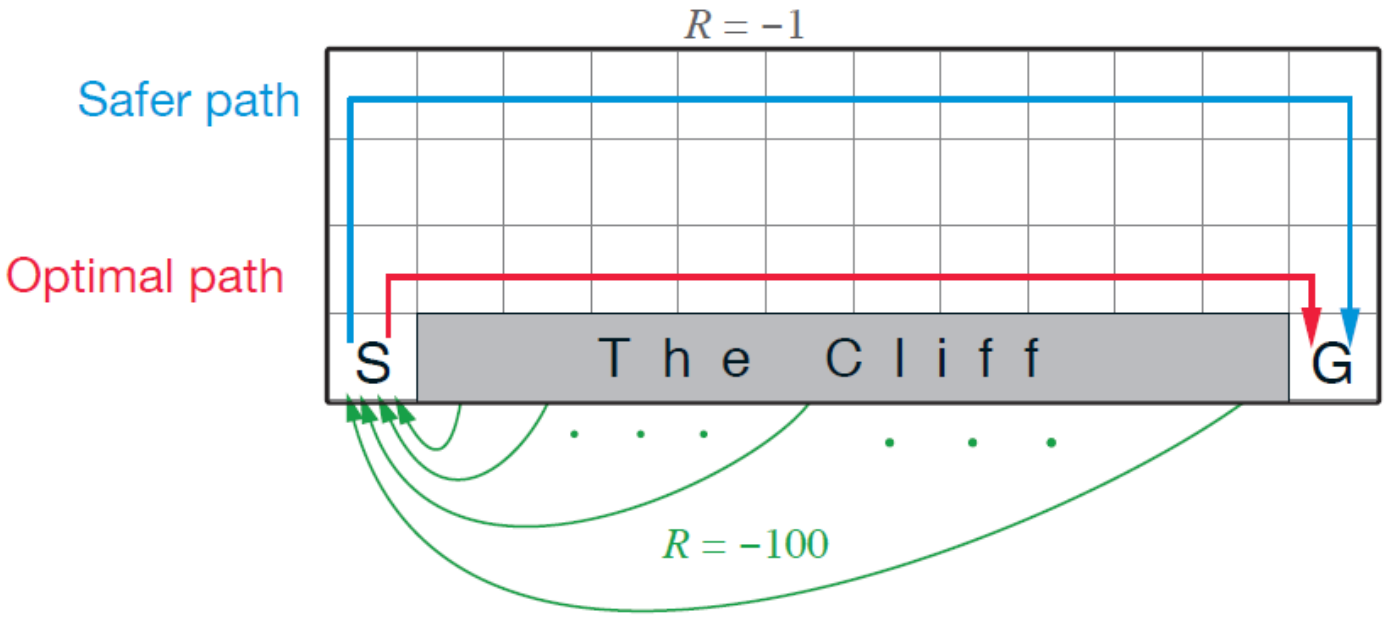
Learning from someone else's data

Question	Distinction
Are we using a transition model in the update?	Model-based / model-free
Can we gather more interaction?	Online / fixed-data offline
Does behaviour match the target policy?	On-policy / off-policy

Q-learning can use transitions from another behaviour policy. This does not supply missing state-action experience.

Question: If an action never appears in the dataset, what evidence updates its tabular Q entry?

Cliff walking: predict before seeing the result

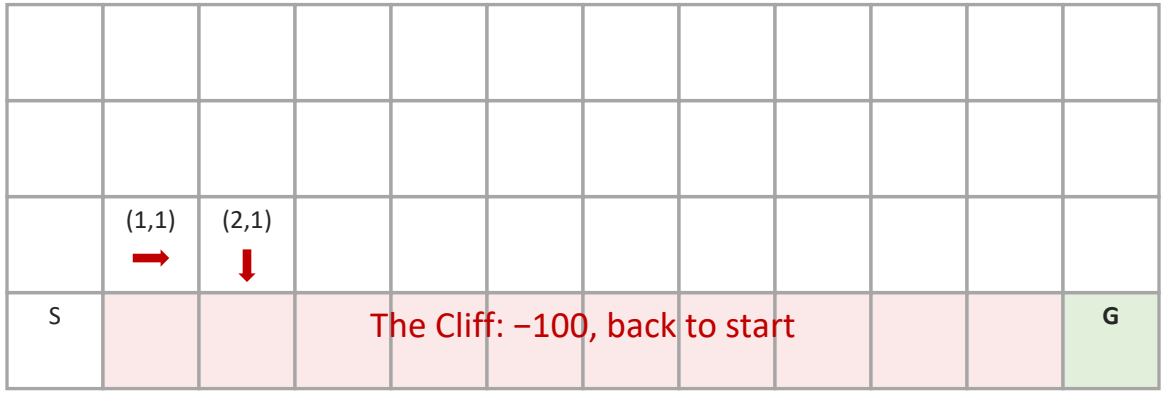


A different task: deterministic movement; four actions; $\gamma = 1$; $\varepsilon = 0.1$.

- Ordinary transition: −1, including arrival at the goal.
- Fall: −100 and return to start; **the episode continues**.
- Only the goal terminates.

Vote: Which path would you expect from each method? Which earns more reward while exploring?

Work a target beside the cliff



Coordinates (column, row): bottom row 0, left column 0

Observe **(1,1), right, −1, (2,1)**; the selected next action is **down**.

$\gamma = 1,\ \alpha = 0.5,\ Q((1,1), \text{right}) = -20.$

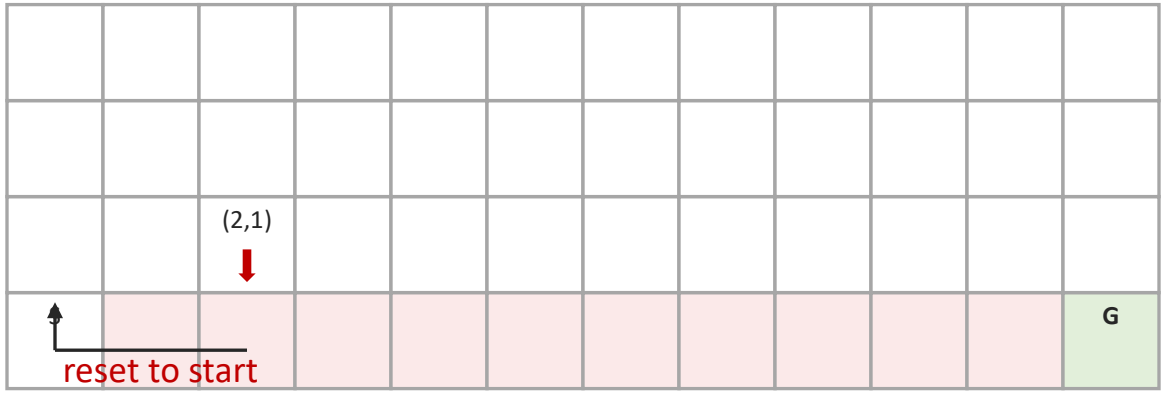
At (2,1):

$Q(\text{down}) = -113;\ \max_a Q(a) = Q(\text{right}) = -10.$

Compute each target and the updated $Q((1,1), \text{right})$.

- SARSA: $U_t = -1 - 113 = -114$; updated Q = **−67**.
- Q-learning: $U_t = -1 - 10 = -11$; updated Q = **−15.5**.

A fall resets the state, not the episode



Now execute the selected **down** action: observe **−100**, next state **start**.

Q-learning's target:

$$U_t = -100 + \max_a Q(\text{start}, a)$$

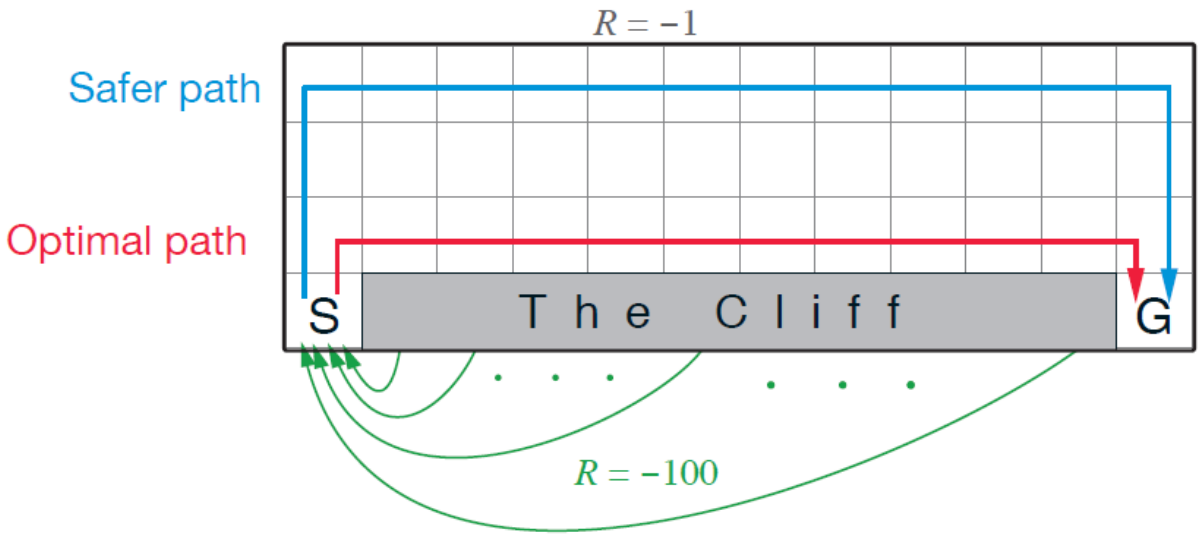
SARSA's target:

−100 plus Q at the next action selected from start.

Question: Is the continuation zero?

No. The goal, not a fall, terminates this task.

Compare fixed policies around the routes

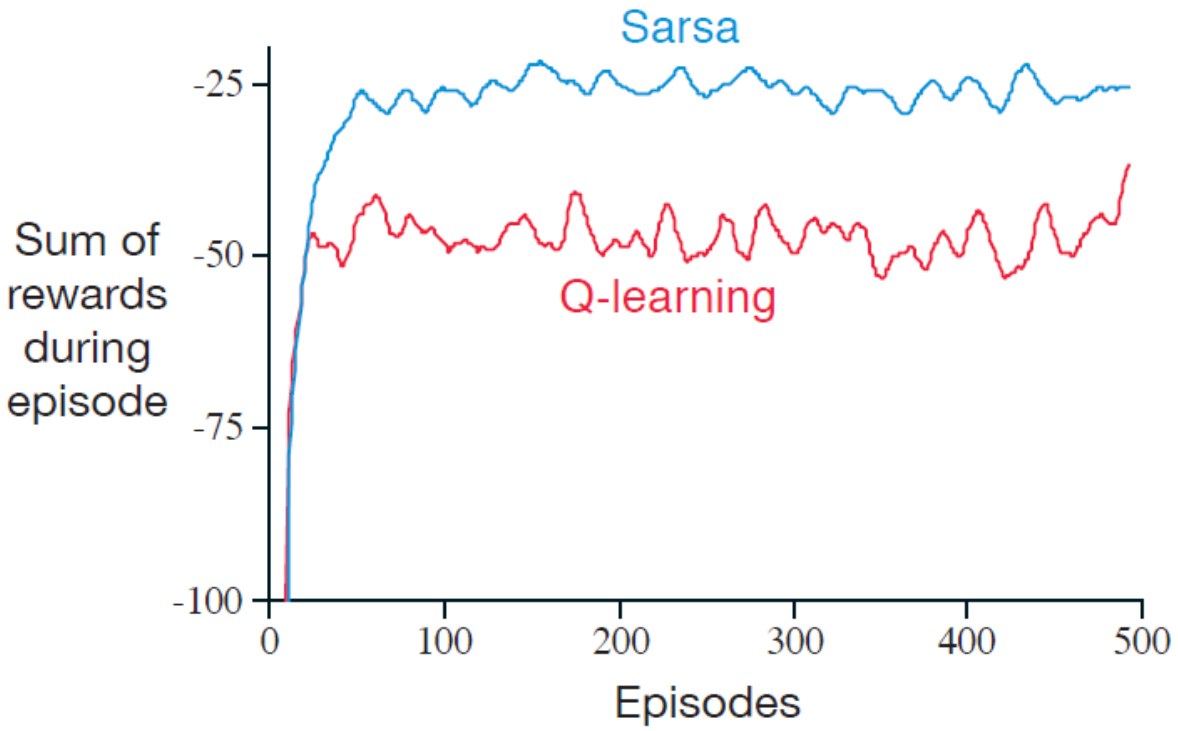


Fixed-policy expected returns from start. Exact model calculations for the policies below; $\gamma = 1$, $\varepsilon = 0.1$.

Policy	Expected episode return
Epsilon-greedy around the edge route	≈ −52
Epsilon-greedy around the top route	≈ −22
Greedy edge route, no exploration	−13

A shorter greedy route can be worse when exploration continues.

What did the learning curve measure?



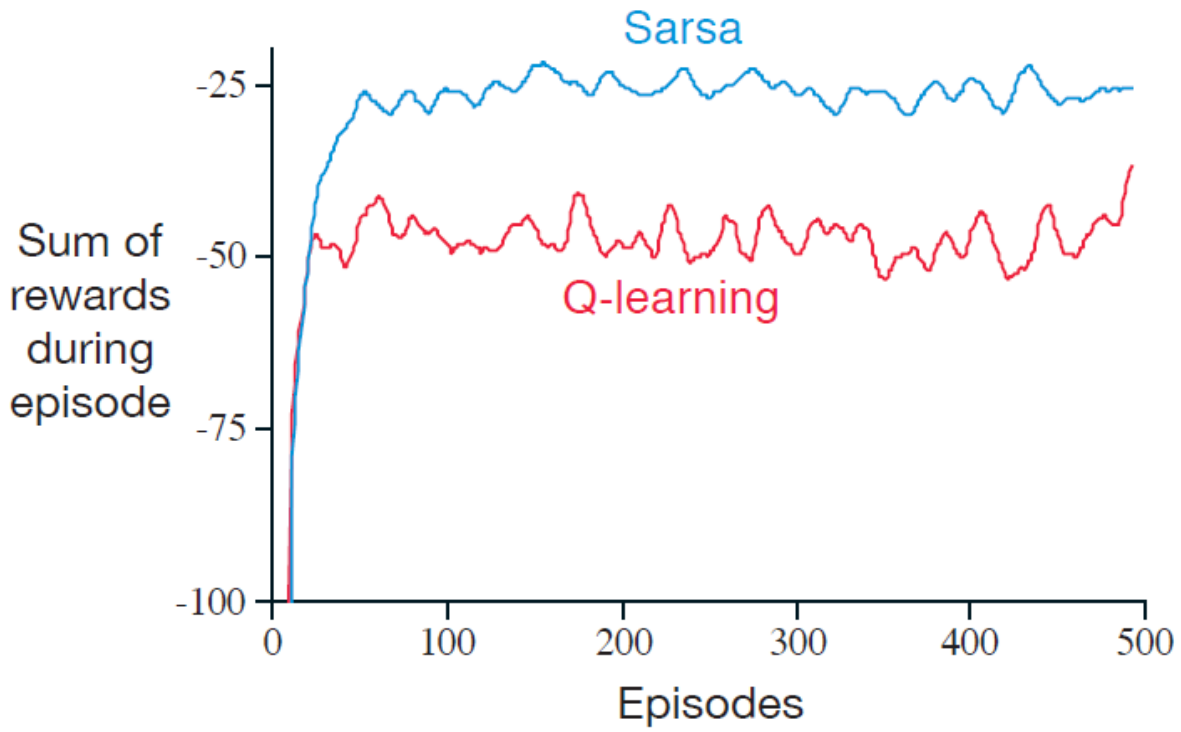
Sutton & Barto, Example 6.6: reward during exploratory learning, ε = 0.1.

Which question are we asking?

- How much reward is collected while learning?
- How good is the final greedy policy in separate evaluation?
- How many falls occur during learning?

A simulator where only the final policy matters; a robot learning near stairs; a policy trained in simulation for deployment. **Would you report the same measure?**

Regret: the price of learning



Sutton & Barto, Example 6.6: reward during exploratory learning, ε = 0.1.

- **Regret:** the reward lost while learning, compared with acting optimally from the start.
- Here the optimal path earns −13 per episode. Each episode's shortfall is −13 minus the curve; regret adds these up.
- Q-learning learns the optimal path but keeps paying for exploration: more regret than SARSA here.

A formal treatment comes with bandits.

What if ε decays, or moves can slip?

Discuss: in each case, which method's answer changes, and why?

- **Decaying ε:** exploration costs less over time, and SARSA's preferred path moves toward the edge. Under the convergence conditions both approach an optimal greedy policy; expected episode return approaches −13 as exploration vanishes, though occasional poor exploratory episodes can persist.
- **Slippery moves** (like our grid's 0.8 / 0.1 / 0.1): falling becomes a risk of the environment, not only of exploration. q^* itself changes, so even the optimal greedy path may move away from the edge, depending on the slip probability and the cost of a fall.

Debug one shared control loop

For each bug, name the mismatch and repair it.

Bug	What happens?
SARSA chooses A_{t+1} for its target, then draws another action to execute.	?
Q-learning always executes the maximizing target action instead of its intended epsilon-greedy behaviour choice.	?
A goal transition includes γ times an initialized terminal Q value.	?

Diagnose what is fluctuating

Observation	What would you check?
Some actions have no data	State-action visits and exploration
Good greedy evaluation, poor training reward	Exploratory cost; whether evaluation freezes learning
Episode returns fluctuate	Action/environment randomness; compare Q and policy changes separately
Q estimates keep changing substantially	Step sizes, target correctness, visitation and changing policy
One run looks better	Repeated runs, tuning effort and interaction budget

Coverage and suitable step sizes matter. A noisy return curve alone does not diagnose nonconvergence.

A meaningful project comparison

Example question: Which learner incurs fewer falls during a fixed amount of cliff interaction?

Item	Example plan
Compared methods	Q-learning versus SARSA baseline
Primary metric	Number of training falls within the interaction budget
Secondary question	Frozen greedy-policy return, stopping at the goal or after 200 transitions
Per-run budget	10,000 environment transitions per method
Repetitions	Three independent training runs per method
Separate evaluation	20 rollouts per trained policy from start; no updates, $\varepsilon = 0$; report how often the cap is reached

Same number of episodes need not mean the same amount of interaction.

Choose your baseline, metric and budget

Eight minutes, with your team or a partner: use a candidate project, or use cliff walking if needed.

1. What question are you trying to answer? What does the agent control?
2. What simple baseline would make the comparison informative?
3. What is the primary metric, and when is it measured?
4. What interaction/computation budget and repeated runs are feasible?
5. What result would *not* support your claim?

Four minutes: share and challenge two or three plans.

Note for Fall26: Do this at home by yourself

From tables to generalization

- Specify the policy represented by the target.
- Distinguish learned values, acting behaviour and the performance measure.
- Check coverage, terminal handling and comparison budgets.

Next: a separate estimate for every state-action pair is impractical in large spaces. How can experience generalize?